万级实时推理的商品领域Agent实践思考和总结

原创 商品中心技术团队 大淘宝技术 2026年5月25日 01:16 浙江 55人



过去几年商品域为应对AI化挑战，构建了"事件驱动的Function-Centric Agent架构"。该架构采用两层设计：上层为业务场景workflow编排层，下层为统一能力供给层，通过AIFunction标准化封装工具和领域知识。系统整合了显性事实、关联情景和隐性经验三类商品知识库，实现了在离线业务流程统一，并基于商品事务事件实现高效实时推理。目前已在商品属性、卖点等核心场景落地，覆盖亿级商品，显著提升信息完整性和搜索转化率，新需求开发周期缩短至1周/人，为商品智能化从"单点提效"迈向"系统自治"奠定基础。

01 引言

随着大语言模型的兴起，商品域在商品理解业务中引入大模型能力，对商品卖点、规格等核心链路进行升级改造。早期沿用T+1离线批处理的生产思路，已在多个场景取得了较好的前台效果。

在这个过程中我们在不断思考几个问题：

（1）AI化是未来很长一段时间的主线，商品域拥有最全的品类和商品供给，工程侧应该储备什么样的技术基建能力，能够立足当下解决业务实际问题，并面向未来持续演进？仅在离线链路跑通几个案例是远远不够的，数据资产和能力的复用必须作为长期目标来对待。

（2）AI化这一抽象命题在商品域如何拆解为可落地、可演进的技术路线？

（3）当前工程使用AI更多的是离线链路，且商品模型各异，是否从离线推理走向实时推理？若到SKU粒度是否能够支撑？当前商品的数万级的写入变更， DB上数据行的变更峰值至少为数十万，是否有有效的方案能解决推理成本问题？大模型生产的数据怎么评价？

（4）在业务上，SKU化的背景和趋势，已经提了这么多年（据材料记载SKU化从2015年左右就有陆续提出），能不能通过agent的思路在商品领域有更完整的解决方案？

02

为什么是商品Agent?

在 2025 年初的大模型工程实践中，业界已探索出多种将大模型能力落地的路径，包括：

- **Prompt Engineering（提示工程）**：适用于简单、静态任务，但难以应对复杂逻辑、多步骤决策或与外部系统交互；
- **RAG（检索增强生成）**：有效解决知识时效性与幻觉问题，但本质仍是“问答式”单轮交互，缺乏主动规划与执行能力；
- **Fine-tuning（微调）**：适合特定领域高精度任务，但成本高、迭代慢、泛化能力受限，且无法动态调用工具；
- **MaaS（Model-as-a-Service）**：提供标准化 API，但将业务逻辑与模型强耦合，难以支撑端到端闭环；
- **Function Calling / Tool Use**：允许模型调用预定义函数，是 Agent 的关键组件，但本身不包含状态管理、目标分解或长期记忆机制。

这些范式在各自场景中仍有价值，但**面对商品域这类高复杂度、强标准化、需持续演进的业务系统，它们均存在“能力碎片化”与“工程不可持续”的瓶颈**。例如，仅靠 RAG 无法自动完成“采集小红书站外知识 → 调用统计算法策略 → 分析属性在当前类目下消费者的提及率 → 生成决策属性建议和理由 → 提交审核”这一多跳流程；仅靠微调无法适应每日新增的品类规则或平台政策变化。

而 **Agent 正是在这些基础能力之上构建的更高阶抽象**：

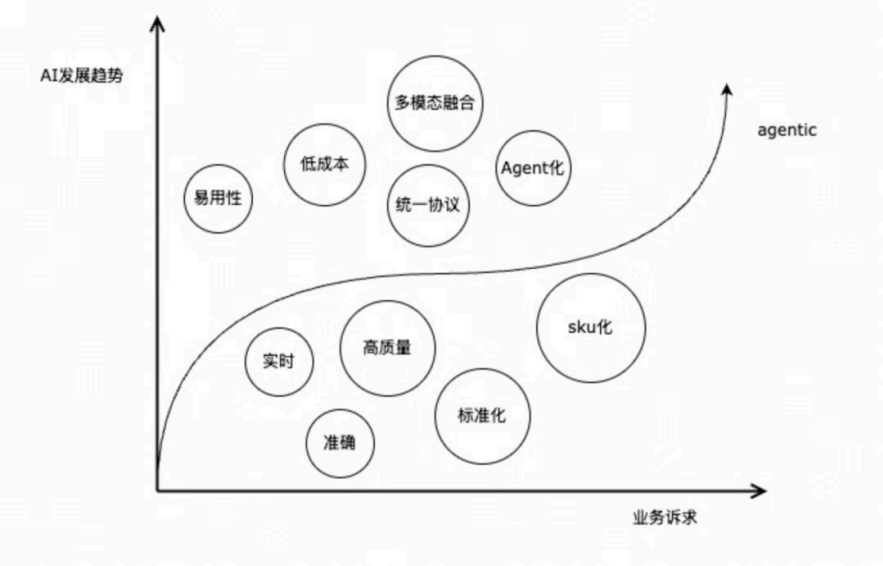
- 它**内嵌了 Function Calling**，实现与现有系统（如商品中心IC、类目中心本地数据包、订单离线数据资产ODPS）的安全集成；
- 它**天然融合 RAG 机制**，通过向量记忆或知识库检索获取上下文相关事实（如类目属性的定义）；
- 它**可结合微调模型作为子能力**，但整体架构保持松耦合（如算法的专用商品图片主体识别模型、算法专用商品图片提取属性模型）；
- 它**超越 Prompt Engineering 的静态交互**，具备目标驱动的动态推理与回溯能力。

更重要的是，**2024—2025 年大模型原生支持 Agent 能力的技术拐点已经到来**：

- 主流大模型已原生支持 ReAct、**Plan-and-Execute**、**Self-Reflection** 等推理框架；
- 开源生态（如 LangChain、LlamaIndex、AutoGen、CrewAI、OpenDevin）提供了成熟的 Agent 编排、记忆管理、工具注册与多智能体协作基础设施；
- 工程可观测性工具开始支持 Agent 行为追踪与评估，使调试与治理成为可能。

因此，**选择 Agent 并非否定其他范式，而是以 Agent 为“操作系统级”容器，将 Prompt、RAG、Function Calling、Fine-tuned 模型等能力有机整合为可工程化交付的智能单元**。

它既继承了已有技术的成果，又解决了其在复杂业务场景中的组合性、状态性和可持续性短板。



商品域结合Agent这一新的技术趋势，我们认为搭建商品领域Agent是实现商品智能化从“单点提效”迈向“系统自治”的关键跃迁，不仅是技术架构的升级，更是对商品数据生产与消费范式的重构，也正是工程侧在 2025 年坚定选择 Agent 作为 AI 化主路径的根本原因。因此，过去一个季度，商品中心围绕“商品Agent”开展初步建设，探索将大模型能力以可控、可集成的方式嵌入商品生命周期链路。重点围绕搜索/详情参数&摘要的智能化建设，支持更清晰、一致、可比的商品信息呈现，从而辅助消费者更高效地获取关键决策信息。

03 商品Agent架构建设

落地商品 Agent 这类面向具体业务领域的 AI Agent时，远不止“接入大模型 + 写 Prompt”那么简单——真正实现其可用、可靠、可提效、可运维，需要系统性地完成一系列工程化工作

框架选型

2025年被称为Agent元年，集团内外“涌现”了非常多的智能体框架，涵盖大模型Agent的方方面面，包含workflow编排、工具集成（MCP）、memory管理等，更有CrewAI、MetaGPT这样的多Agent解决方案。**通过广泛评估和调研，我们最终决定选择轻度“耦合”spring-ai-alibaba来构建商品域Agent应用。毕竟在集团成熟的Java生态下，新的技术栈势必带来更多的额外复杂度，可维护性差并且与现有系统的集成难度显著升高。**但未来也会向多应用类似微服务的架构演进，引入开源社区丰富的Agent组件如deepeval做评测、deepresearch做事实性验证等。附上在调研阶段，对一些相关框架病结合商品特性做一个的简单对比小结：



架构设计

架构设计主要围绕以下几个命题来进行：

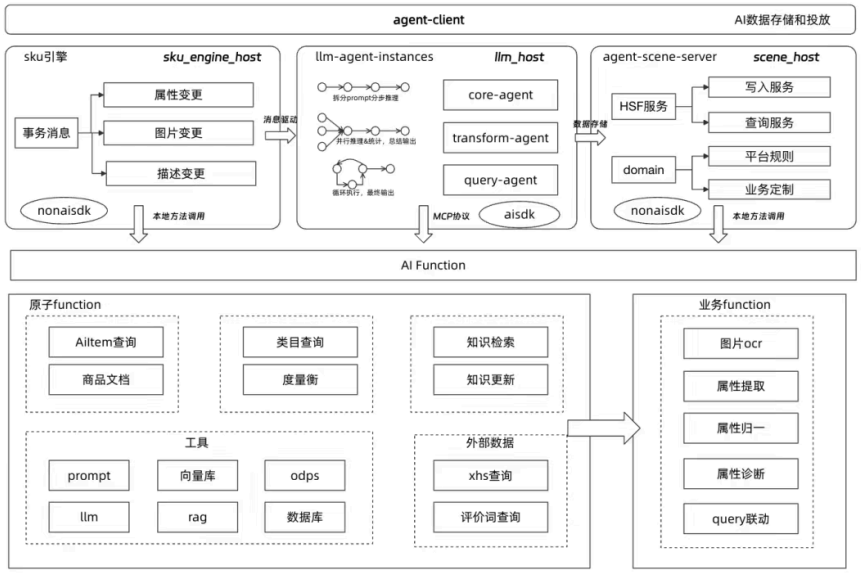
（1）面对具象的各异的前台业务场景，需要重点建设哪些组件来降低新场景的开发成本。

（2）商品领域知识如何融入大模型调用流程？

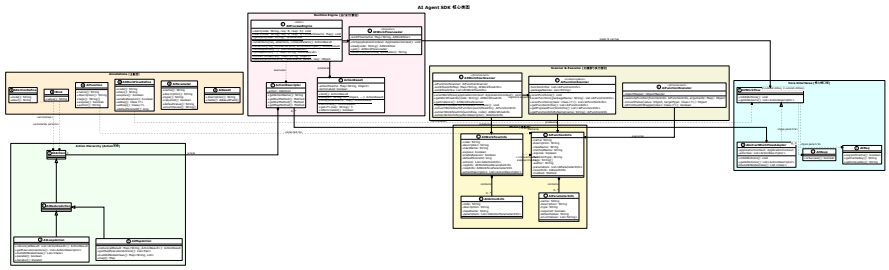
（3）要实现成本的最优，我们认为需实时和离线结合使用，那如何设计workflow及其调度 实现在离线处理逻辑一致？

（4）是否能走向实时推理？

结合大模型agent的几个核心（抽象）要素，我们认为Agent的应用架构应该足够简单和灵活，应采用类似大数据处理引擎的视角来看待Agent架构。因此我们将商品 Agent 抽象为两层结构：**上层是面向业务场景的 workflow 编排层，下层是统一的能力供给层，两层之间通过抽象的AIFunction接口来交互。**



我们希望底层通过 **AIFunction** 对各类工具和领域知识进行标准化封装，向上提供结构清晰、语义明确、调用稳定的函数接口，从而支撑上层灵活、高效地构建智能流程。为此也开发了轻量的aiagentsdk，提供一组简洁易用的注解，如：@AIWorkflow、@AIAction、@AIFunction、@AIParameter、@AIResult、@AIResultField等。在注解和切面的增强下，所有function可轻易转换成一组高质量的mcp工具，供问答型agent自由调用。



- AIFunction：灵活高效可复用

AIFunction定义规范

使用agent-sdk中定义的注解@AiFunction进行声明，其中参数使用@AiParam进行标注。AiFunction中包含如下字段：

基础必备字段（最小集合）：这部分字段须统一行业规范，如OpenAI Function Spec中规定的字段名和用途，具备健壮性。

- `name`：（可选，默认类/方法名）function唯一标识或显示名
- `description`：（必需）一句话能力描述，主要给AI/开发/文档看
- `parameters`：不需要必填，可通过反射自动通过@AiParam进行推导
- `returns`：（可选）返回值类型/含义说明

- `expose`：（可选）默认false，标明当前function是否对外暴露

扩展字段：

- `tags`：能力标签，便于多维分组检索，比如"llm"、"rag"、"tool"、"java2python"、"电商数据"、"memory"
- `author`：维护人，便于治理
- `sideEffect`：布尔型，是否有副作用（如写库、发消息、变更状态等）
- `timeoutMs`：推荐执行超时时间（AI/流程调度友好）
- `deprecated`：是否废弃

AIFunction访问规范

function作为整体架构约束，提供workflow、AI自规划两种模式下统一的访问方式会使得架构更统一和完整；但workflow是提前编排好的一组节点，节点内function也是固定的，基于统一入口的调用方式会使得常量或者枚举类泛滥，不好管理。比如下面这样的代码片段：

```
functionRegistry.invoke("ITEM_QUERY");
functionRegistry.invoke(FuncEnum.ITEM_QUERY);
```

因此必须在架构SDK层面对此进行约束和改良。我们要求调用function时，必须按照{Registry}.{DomainRegistry}.{FunctionClass}.{FunctionName}的方式进行调用（仿照老IC客户端的模式）。这种方式若人工实现，势必带来很重的负担，每次新增Function都必须去对应的分组中进行手动声明。

因此希望通过lombok的模式，对注解进行编译期扫描，自动生成引用，来完成以下这样的链式调用：

```
registry.item().query().invoke(params)
```

这样在代码层面、编译器、运行期架构拥有一个统一、完整的视图。

AIFunction具体实例

一个典型的function声明如下：

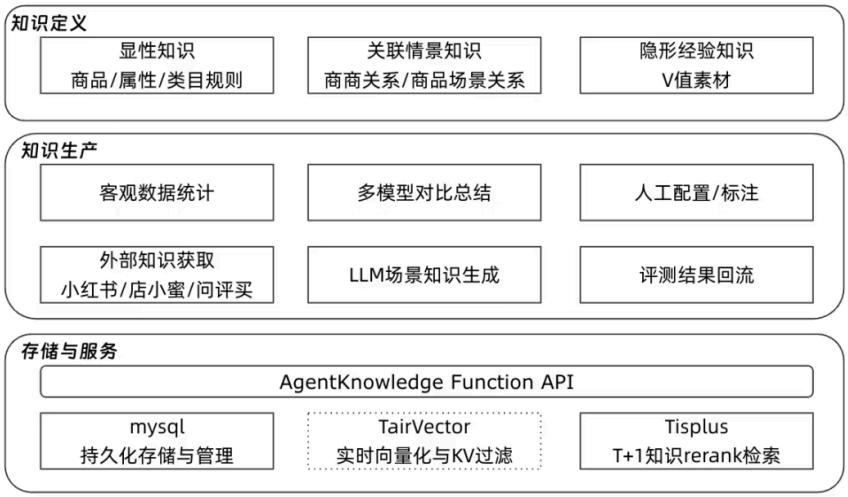
```
publicinterfaceItemQuery {
    String QUERY_AI_ITEM_BY_ID = "queryAiItemById";
    @AiFunction(name = QUERY_AI_ITEM_BY_ID,
        description = "查询商品信息",
    expose = true)
    @AiResult(description = "商品数据")
    AiItemDO queryAiItemById(@AiParameter(name = "itemId", description
}
```

其中返回值AiItemDO对HSF接口返回的ItemDO做了大量精简和字段含义的标注，再通过商品文档知识库的辅助，帮助大模型在ReAct的loop中准确选择工具并得到恰当的解释。

回到前面提到的开发成本的命题，在这样的设计下，新的场景可以快速复用已有的function能力。如“workflow产出的结果也可以使用通用的Repo工具进行context和推理结果的存储（OdpsWriter和在线StorageWriter）。

- 商品领域知识库：从定义、生产到服务

知识库架构图

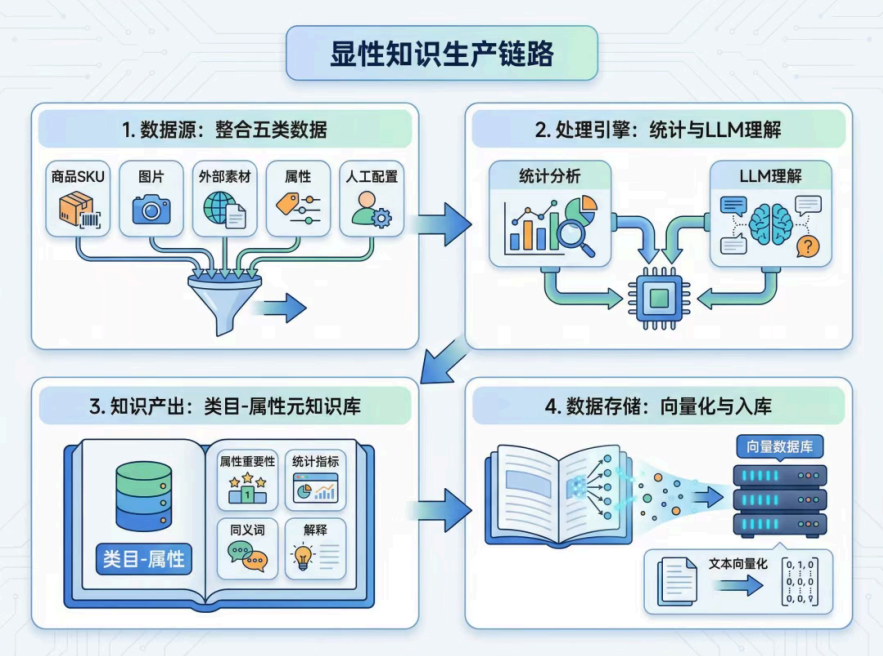


知识定义

1. 显性事实知识
    a. 在商品领域是对“显卡的GPU品牌是XXXX”、“显卡类目同一SPU下不同SKU差异较大”的客观描述。
    b. 关键使用场景：运营决策（试跑阶段的数据有效的沉淀一些行业或者品类知识，后续可以用于做类目灰度的优先决策）、prompt增强、数据清洗和标准化
2. 关联情景知识：
    a. 这层知识将商品与商品、需求场景连接起来，未来可以考虑与人的关联。如：商品-商品关系、商品-场景。
    b. 关键使用场景：主配件
3. 隐性经验知识
    a. 知识的顶层，难以被量化，但也最能建立信任和情感连接。通常来源于大量用户的使用经验、专家的评测和品牌的文化沉淀。
    b. 关键使用场景：商品卖点、参数说明

知识生产&应用

显性知识生产



显性知识生产链路围绕“类目–属性元信息”构建，整合五类数据源：商品SKU主数据、商品图片（如主图）、外部素材（如小红书笔记和用户评价）、原始商品属性以及人工评测与运营配置，分别输入统计分析模块和LLM理解模块：前者计算填写率、SKU数量、PV对等统计指标，后者从图像和文本中提取属性、识别用户关注点并生成同义词与解释说明；二者输出与人工校验结果共同汇聚为结构化的类目–属性元知识库，包含决策属性重要性、必填性、枚举/输入类型、填写率、TOP值、商品/SKU标识、同义词、客观解释、SKU图上传率、属性覆盖数、平均SKU数、优级SKU占比、属性PV对及单图平均提取属性数等元信息；最终，知识库中的文本字段（如同义词、解释）被向量化存入向量数据库，支持高效语义检索。

c/cp/cpv知识样例：

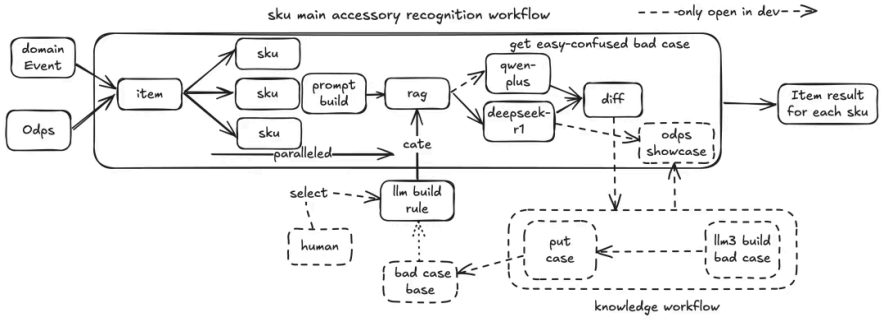
商品属性名rerank的结果


```
{
  "data": [
    {
      "cateName": "显卡",
      "propertyName": "GPU品牌",
      "topItemCntValues": [
        "NVIDIA/英伟达",
        "AMD",
        "intel/英特尔",
        "凌久",
        "丽台"
      ],
```

```
      "definition": "GPU品牌 是指显卡中使用的图形处理单元（GPU）的制造商或品牌，
    },
  ...
  ],
  "success": true
}
```


情景知识提取

主配件场景在bad case结果集总结出的知识库中当前在10个类目的近10000条案例中总结出53条规则：



主配件场景badcase结果总结数据


```
[
    {
        "uuid": "7c2fb326-5226-4185-b7e0-c32219f65936",
        "contentRaw": "{\"cate\":\"笔记本电脑\",\"examples\":[\"硬盘容量、
        "customConfig": {
            "cate": "笔记本电脑"
        },
    },
  ...
]
```


隐性经验知识

V值场景化主观描述，知识样例

SKU智能筛选场景


```
{
    "core_scenario": "为现代家庭客厅提供兼具美观、实用与空间适配性的茶几选择，满
    "structured_filters": [
        {
            "attribute_name": "长度",
```

```
            "knowledge_needed": true,
            "values": [
                {
                    "value_name": "1.2米",
                    "scenario_translation": "小户型优选，节省空间不拥挤。"
                },
                {
                    "value_name": "1.4米",
                    "scenario_translation": "适合两人沙发，比例协调显精致。'
                },
                {
                    "value_name": "1.6米",
                    "scenario_translation": "适配三人沙发，日常使用刚刚好。'
                },
                {
                    "value_name": "1.8米",
                    "scenario_translation": "大客厅标配，摆茶具零食更从容。'
                },
                {
                    "value_name": "2.0米",
                    "scenario_translation": "宽敞客厅之选，聚会待客有面子。'
                },
                {
                    "value_name": "2.2米",
                    "scenario_translation": "大平层理想尺寸，气场十足显档次。
                },
                {
                    "value_name": "2.4米",
                    "scenario_translation": "别墅客厅优选，搭配L型沙发更和谐
                },
                {
                    "value_name": "2.6米",
                    "scenario_translation": "超大空间专属，彰显大气家居格调。
                },
                {
                    "value_name": "2.8米",
                    "scenario_translation": "豪宅级配置，打造客厅视觉焦点。'
                }
            ]
        ...
            }
    ]
}
```

知识服务

在知识服务体系建设过程中，我们投入了大量精力深入探讨如何高效、灵活地存储和管理多类型知识。面对多样化的知识形态（如结构化元数据、半结构化规则、非结构化文本等），系统不仅需要支持高可靠的知识写入，还需满足多维度动态检索与基于语义的精准召回需求。为此，我们设计并实现了一套统一而可扩展的知识存储模型与服务体系。

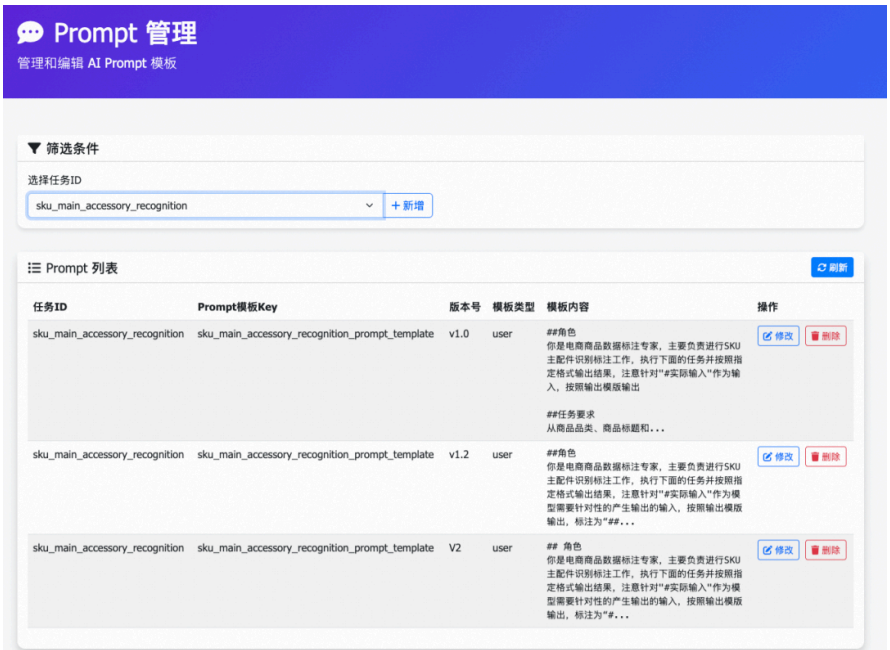
该体系对外提供标准化的知识服务接口，包括**写入（save）** 和 **读取（query / count / pageQuery）** 核心能力，便于上层业务流程通过函数调用（function）无缝集成。在底层存储架构上，采用两层异构存储策略，兼顾实时性、持久性与语义检索能力：

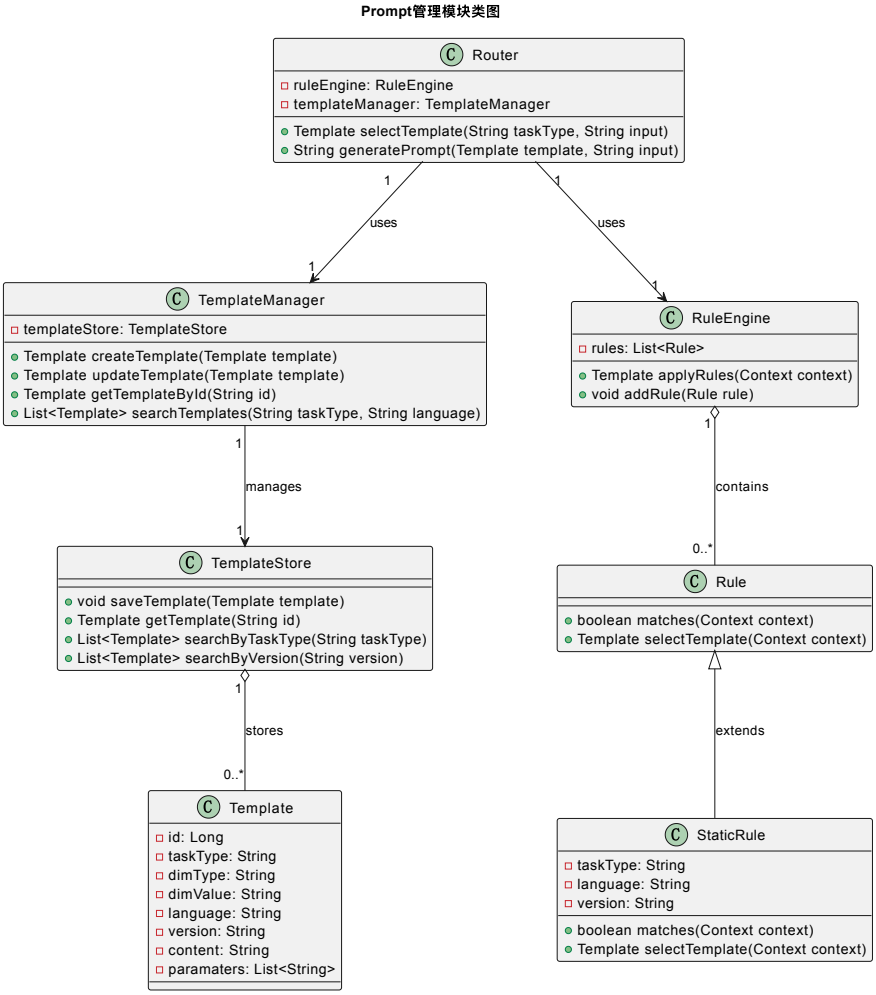
- **MySQL**：作为主持久化存储，保障知识数据的强一致性与事务完整性，适用于结构化元信息的长期保存；
- **TisPlus**：提供批量向量化处理与大规模KV存储能力，适用于离线构建高维索引、深度语义挖掘及历史知识回溯。

通过协同工作，系统既能支撑高频、低延迟的在线知识调用，又能满足复杂语义理解与大规模知识分析的需求，为上层智能应用（AI决策属性排序、筛选&SKU参数提取场景运营决策等场景）提供坚实、灵活且可演进的知识底座。

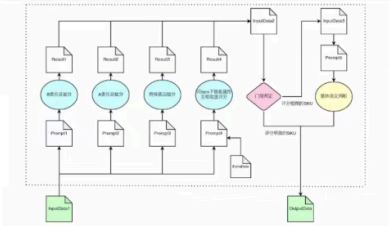
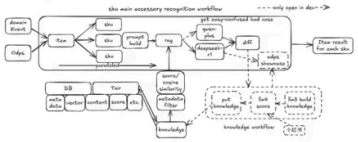
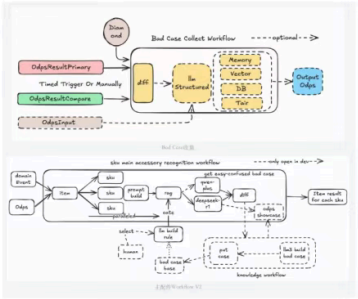
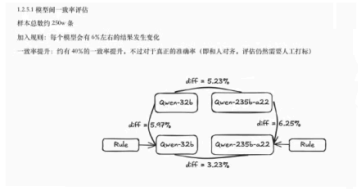
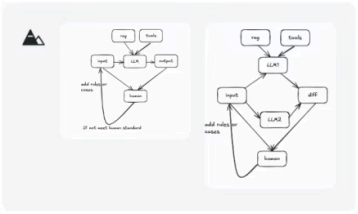
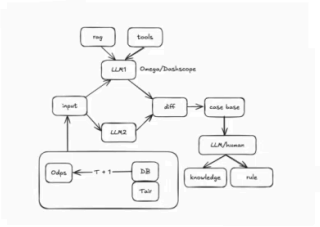
- 提示词工程：经验沉淀为模版

在提示词技术方面，一开始就提供了通过（workflowname，templatekey，version）三元组持久化和读取prompt的function，开发可以通过不断调休提示词版本，来提升数据产出的质量，同时开发在可以开发工作台低成本的维护提示词。



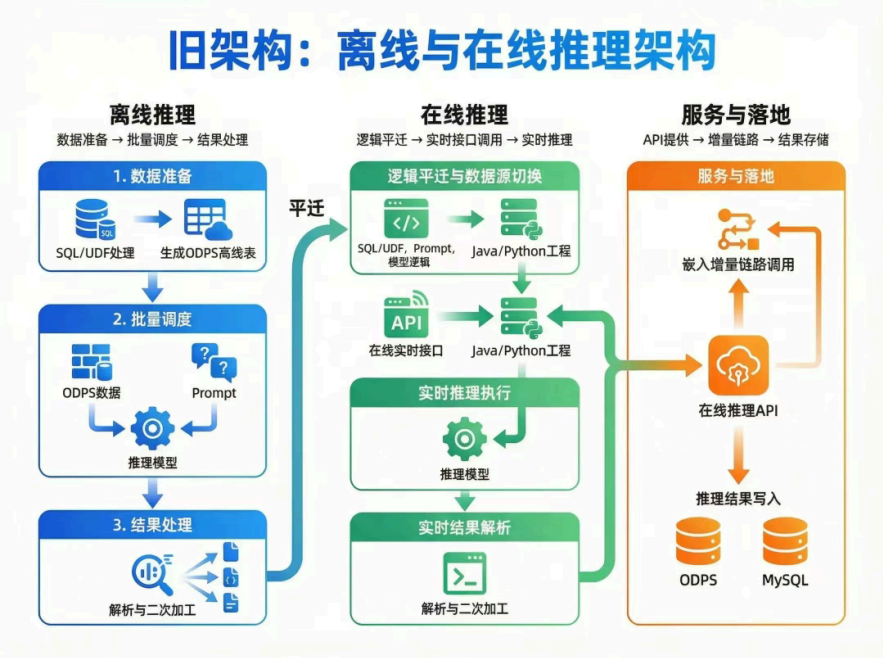


后面在对不同workflow的长期调优过程中，我们总结出了几种最常用且有效的提示词使用技术，即**架构图中体现的几种：长prompt拆分后的分步推理、单prompt循环执行、并行推理&统计+总结等**。这几种不同的推理流程在sdk中使用不同类型的Action节点来支持，如Map节点、Reduce节点、Loop节点等。这些宝贵的实践，主要由我们年轻的实习小师弟们在实习阶段的实验场景：

<table>
<tr><th></th><th>Prompt拆分案例：无效SKU识别</th><th>Prompt调优+总结案例：SKU主配件</th></tr>
<tr><td>实验过程和结果</td><td>一阶段：单prompt并且版本优化
二阶段：拆分成下图，多prompt
 
实验结果数据：
抽选了四个类目（电脑硬件/显示器/电脑周边、居家日用、零食/坚果/特产、标准件/零部件/工业耗材）共57815条SKU数据，分别使用Prompt_v2.1、Prompt_v2.2以及Prompt拆分链路进行识别推理，将结果对比如下：
<table><tr><th></th><th>识别率（无效SKU识别量/推理量）</th><th>正确率（正确识别量/无效SKU识别量）</th></tr><tr><td>Prompt_v2.1</td><td>159 / 57815 = 0.28%</td><td>152 / 159 = 95.6%</td></tr><tr><td>Prompt_v2.2</td><td>2135 / 57815 = 3.69%</td><td>964/ 2135 = 45.2%</td></tr><tr><td>Prompt拆分链路</td><td>1055 / 57815 = 1.82%</td><td>973 / 1055 = 92.2%</td></tr><tr><td>期望值</td><td>约2%</td><td>>90%</td></tr></table>可以看出，Prompt拆分链路的推理结果具有适中的无效SKU识别率以及较高的正确率。</td><td>一阶段设计：
 
二阶段设计：
 
实验结果数据：
  </td></tr>
<tr><td>总结</td><td>➔ 优势与适用场景
综合调试过程与最终结果，可以得出：相比单个Prompt，Prompt拆分链路具有以下优势：
• 较高的推理稳定性：LLM规则遗忘与混淆的情况减少，多次推理综合也能够避免单次推理的偶然性误差。
• 较高的可维护性：中间结果可见能够快速定位问题，各节点的规则调整不会相互影响，对调优友好。
• 更细的调整粒度：单个节点规则调整产生的影响更小，能够从多方面进行效果调优。
Prompt拆分的适用场景有：
• 多维度的评判型任务：按照不同的评判维度对规则进行拆分；
• 流程长且顺序明确的任务：按照流程顺序对规则进行拆分。
➔ 问题
虽然Prompt拆分具有不少优势，但仍存在以下问题：
• 链路长，中间结果处理环节多，响应时间长，消耗LLM推理资源更多；
• 方案制定与实施复杂。
在实践过程中发现，后续还可能通过以下调整来优化效果：
• 设定合适的评分判定门限；
• 仅对被预判定为模糊的SKU进行销售属性互相似度评分，以节省资源、降低平均推理时间；
• 在线链路启用多线程、选用推理速度较快的模型，以降低平均响应时间。</td><td>在 prompt 调优的过程中，无论是 few-shot，还是“规则”，本质上不是在给大模型提供“知识”，而是类似于人和人的沟通，实质上是在对齐“需求”。
举例：在常见的 prompt 工程路径，我们是如何调优的
 
新的可能的范式：
  </td></tr>
</table>

- AIWorkflow：在离线业务流程统一

旧架构实现



为什么旧架构采用上述模式？这主要受限于当时的技术条件与工程能力。具体原因有三：首先，彼时缺乏成熟的在离线一体化数据处理框架，难以统一支撑实时与批量场景；其次，大模型的实时推理成本高、响应延迟（RT）较长，且在线服务通常设有严格的 QPS 限流，难以满足高并发实时需求，因此业界普遍采取“先离线、后实时”的演进路径；最后，业务尚处于探索阶段，亟需快速验证大模型能否带来实际价值。基于上述背景，我们优先构建了基于 ODPS 离线节点与批量推理平台（Omega）的离线推理链路

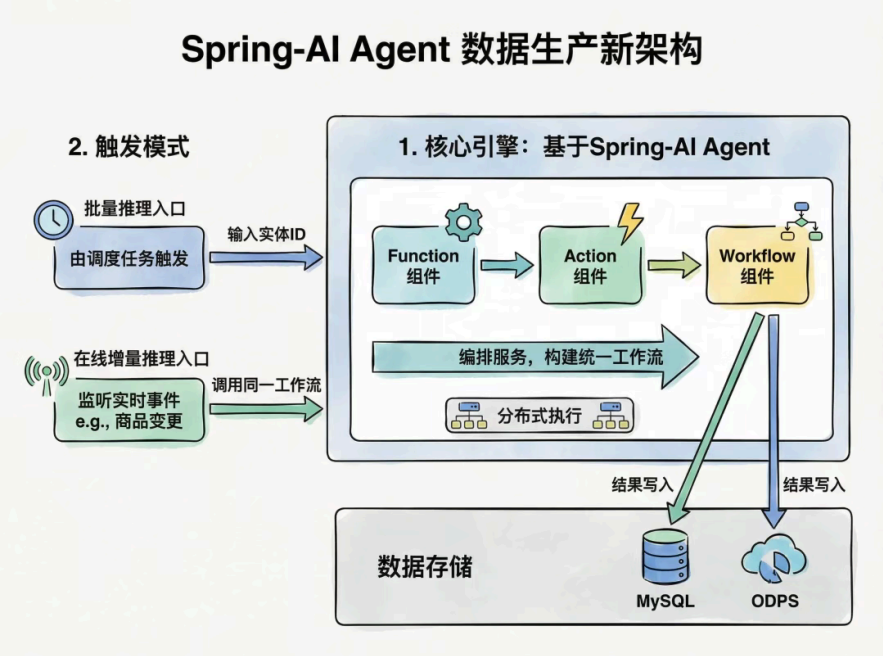
旧架构存在的问题

随着业务发展，我们在 2024 年探索大模型与业务融合的过程中，逐步暴露出原有链路的若干关键问题：

1. 数据处理复杂且维护成本高：批量推理的核心工作量集中在输入/输出数据的加工、解析（如复杂 SQL、UDF）以及离散离线节点的编排与调度。复杂的 SQL 可读性差，离散节点难以统一管理；同时，受上游数据延迟、大促期间资源紧张等因素影响，任务调度稳定性难以保障。
2. 流程扩展性与灵活性不足：面对复杂推理场景（如多轮迭代推理、多模型对比、推理后追加评测子流程等），现有“离线节点 + Omega”模式难以灵活支持，即使勉强实现，也因逻辑耦合、缺乏抽象而难以长期维护。
3. 推理调度存在不确定性：批量任务依赖共享资源池，易受资源争抢影响，导致任务延迟或产出不稳定。虽可申请独占 GPU 资源，但按卡粒度分配易造成资源浪费（申请过多）或处理效率低下（申请不足），最终仍被迫回归共享模式，陷入两难。
4. 在线与离线体系割裂：当前在线与离线采用完全独立的两套技术栈，代码、调度、推理逻辑各自维护。不仅带来高昂的重复开发与维护成本，更难以保证两者在数据处理逻辑上的一致性，严重影响迭代效率与结果可靠性。

针对上述问题，我们在 2025 年通过构建一套更工程化、统一化的智能体（Agent）驱动架构，系统性地实现了在线与离线推理的深度融合与高效协同。

新架构的实现



业务实现在离线统一的方式有多种，例如可借助支持在离线一体的流程编排平台。但综合考虑业务逻辑的复杂性、未来对 Agent 扩展能力的需求，以及对全流程更高透明度和可控性的要求，我们最终选择通过更工程化的方式来实现在线与离线统一的处理。具体而言，我们基于 Spring AI Agent 将业务流程的复杂步骤封装为一个 Workflow，向上屏蔽触发源的差异（如定时调度 vs 实时事件），向下屏蔽计算资源的差异（如单机执行 vs 分布式集群），从而在代码、部署、推理资源和存储层面实现真正的统一。

在核心逻辑层面，我们实现了组件化与标准化：将业务处理过程抽象为三个标准化组件——Function（原子能力）、Action（业务动作）和 Workflow（流程编排）。所有业务逻辑，无论在线还是离线，均封装在同一个“统一工作流”中，由统一的编排服务驱动。这不仅实现了业务逻辑的集中管理，还彻底解耦了原本散落在不同代码库或平台中的实现。同一套代码既能支撑批量跑批，也能处理实时请求，显著提升了复用性与一致性。

在触发机制上，架构设计支持两种入口模式：一是离线批量推理，由调度任务触发，输入为实体 ID 列表，适用于大规模、非实时的存量数据处理；二是在线增量推理，由实时事件（如商品信息变更）驱动，适用于低延迟、高时效的增量更新场景。两种模式共享同一套工作流逻辑，仅入口不同，真正实现“一套逻辑，多端适配”。

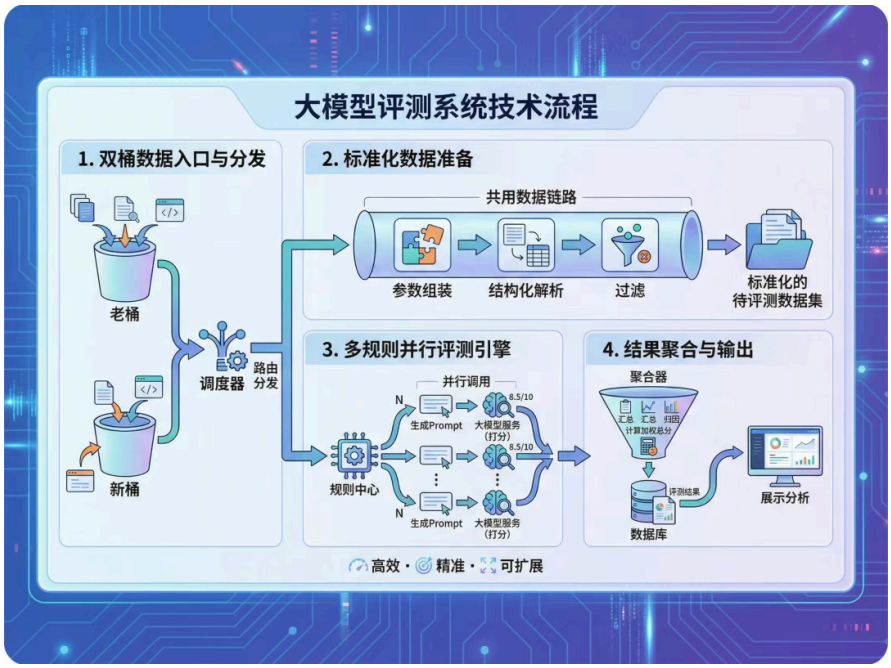
为应对离线场景下的高吞吐需求，Agent 应用内部实现了通用的分布式数据处理能力。基础 Function 支持并行化与分片处理，Workflow 可根据负载动态调整批量处理的并发度与吞吐量，确保离线任务高效稳定执行。

在结果存储方面，我们实现了统一的写入机制：无论是离线还是在线调用，均由统一的存储写入 Function 负责。该机制支持将结果写入 MySQL/，供在线业务系统进行低延迟查询；同时可支持同步落盘至 ODPS，支撑离线数仓分析。双写操作由统一工作流协调，确保数据语义一致、链路可追溯。

通过上述设计，我们不仅实现了在线与离线 AI 推理的深度融合，也为未来引入多 Agent 协作、动态决策等高级能力沉淀了丰富的可复用原子业务组件，为系统长期演进奠定了坚实基础。

- AI生产数据质量评测体系

基于商品侧知识，利用AI实现商品与导购供给的智能化理解（特征识别、属性理解、场景导购等），目标是高质量、规模化、实时化地产出数据。面临挑战包括数据质量、规模化成本、实时性、资源瓶颈、置信度、离线与实时不一致及合规问题。为此，工程和测试同学一起，基于agent应用，构建一套可量化、可归因、可复现的属性摘要大模型自动化评测系统，科学评估新版数据的效果，评测系统流程如下：

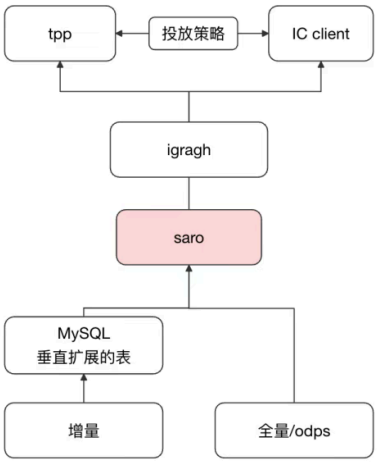


- 实现新桶 vs 老桶召回属性商品的数量、排名等信息的AB对比评测；
- 利用大模型进行语义理解与打分，构建多维度规则体系，覆盖V 值准确性、完整性、可读性、一致性等；
- 可以根据不同的评测场景，配置打分规则集；
- 建设可持续的数据搜索与评测链路，评测归因等数据可视化，支撑决策优化，并确保结果的可追溯性。

	
评测归因后的可视化图	运营也可以针对单品或者线上badcase进行定向实时评测，帮助判断

- 数据投放链路：低成本接入消费

基于商品领域agent生产的数据，通过saro+RDS的存储方案，可以通过调用ic现有的商品查询接口，但需要指定的特殊参数；在搜推广场景，我们也提供相应的tpp和saro能力，可以支持不同的场域或者应用链路接入；当然也接受业务侧自行生产的数据通过ic链路进行投放，下游进行消费。目前投放侧，也有一些投放策略进行选择，如query联动、字段优先级等。



- 事务型商品领域事件：实时推理的关键

在主搜SKU化的背景下，商品的存储模型复杂和迭代给ETL链路增加了理解成本和实施成本，与此同时从商品行到SKU行的变更（从万/s上升到数十万/s）处理给ETL链路带来了巨大的性能和成本的挑战。为了解决前者问题，FY25通过saro平台构建了SKU引擎1.0版本，主搜

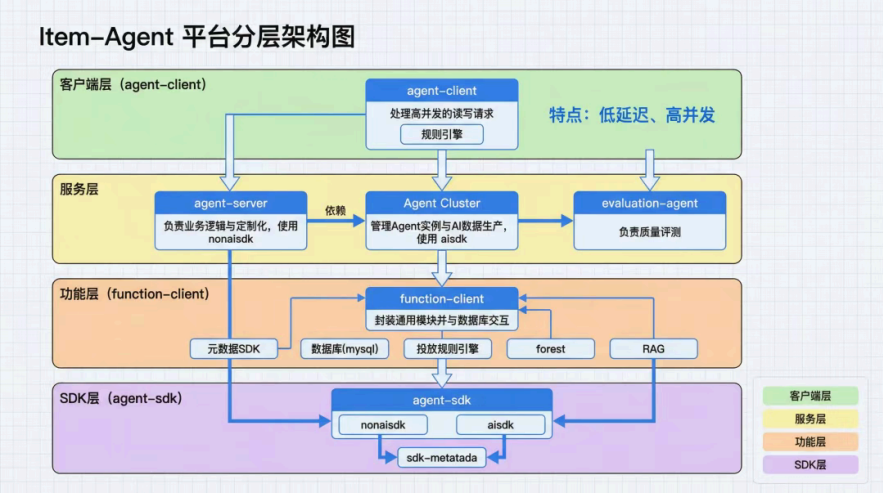
dump不再直接拉商品数据库GALICDB，而是通过saromirror的形式进行交互，IC内部完成存储模型到导购商品或者sku引擎数据字段的加工和处理。

但不管是saro还是精卫这样的数据通道，为上层提供的都是数据表+行变更的视角。而在商品域商家或者平台一次操作往往涉及多个数据表间的事务操作，会造成量级本身的放大；此外多表间特定字段的变更往往在应用层存在一定的链路或模型约束，如A表A1字段变更一定伴随B表B1字段变更，在数据表视角下这类变更需要频繁使用多表JOIN来计算最终结果，进一步加大了计算量（在saro链路上多表join由于A表B表两个字段会单独变更，因此会触发2次JOIN，造成进一步放大）。

为此，今年6月份我们升级了SKU引擎架构，通过将SKU变更降维到商品事务维度，作为ETL链路的数据底座，**同时作为商品Agent推理的驱动源**。我们在精卫链路设计并实现了基于商品ID+事务ID进行数据行变更聚合并转发的事务消息链路。精卫任务中只处理并发送下游关注字段（或模型）的事务消息，将秒级别最终要处理的事务量级降低了一个数量级；下游使用Java应用消费消息并补全数据，最终落到一张以经过增强的全局唯一skuId为主键的异构sku数据表，这张数据表直接DUMP到saro链路无须额外处理即可作为完整的sku数据供搜索消费。以上两步分别消除了一次操作到多行变更的放大和多行变更间的JOIN计算，因此具备极好的性能和低成本；

同时，我们获得了实时的sku素材（包括属性文本、图片等）、商品基础信息等变更驱动源，为接下来的商品实时Agent打下了良好的基础。当前，商品agent通过事务型商品领域事件搭建了多个业务场景的实时推理能力。

应用分层



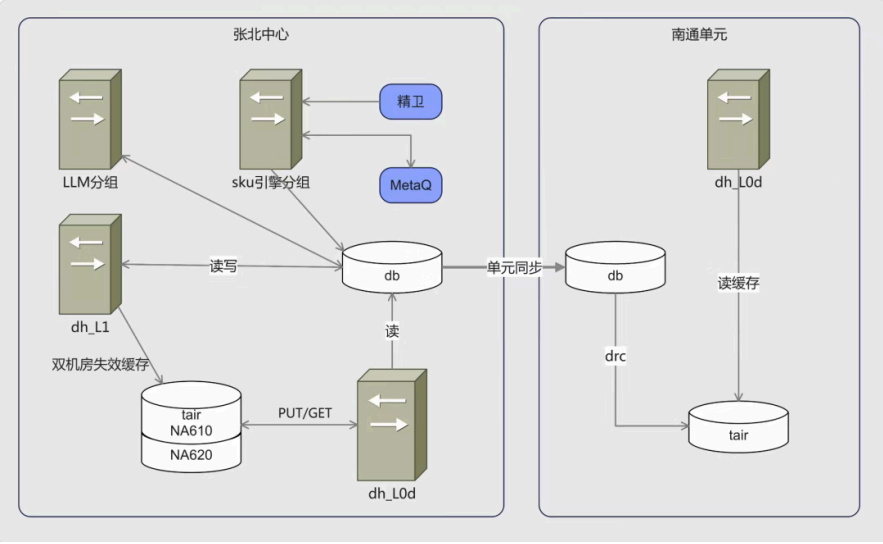
商品Agent系统采用高度模块化、分层解耦的架构设计，以支撑复杂AI能力在商品领域的规模化落地与持续演进。整体架构自上而下形成一条清晰的主干依赖链：

- 客户端层（`item-agent-client`）： 以 JDK 8 兼容的轻量级 SDK 形式，定义对外的标准化能力。
- 服务层，包含3个模块：

- `agent-server`：该层封装商品、属性、素材等核心领域的业务逻辑，通过repository/dao/model 分层实现数据访问与领域建模的分离，并提供统一的应用服务接口；
- Agent 实例层（`item-agent-instances`） 具体agent实例的实现，按业务场景拆分为商品数据加工、商品数据问答、SKU引擎等多个可独立部署的子模块，每个模块内含工作流编排与领域模型；
- 评估客户端层（`item-agent-evaluation-client`） 专注于agent实例的效果度量与 A/B 测试，形成“执行–评估–优化”的闭环

- 功能层（`item-agent-functions`）： 提供 Agent 可调用的原子能力和业务能力，如向量写入、文本解析、属性提取等，所有函数均注册至元数据中心，支持运行时动态发现与编排；
- SDK 层（`item-agent-sdk`），抽象出面向 AI 与非 AI Agent 的统一调用契约；
- 两个公共模块：
  - 事件引擎层（`item-agent-event-engine`） 负责异步消息与事件驱动，保障最终一致性；
  - 管理后台层（`item-agent-admin`） 提供运维管控、定时调度与人工干预能力；

部署架构



1. **SKU引擎分组**：通过商品领域事件，把从商家管理侧以商品粒度的商品信息，转换并映射成前台导购链路需要的SKU粒度，在agent中完成有无SKU、item和SKU状态以及属性文本等商品领域内的逻辑处理；目前针对前台导购场景有主站SKU和门店SKU库。该集群随着主搜切到SKU引擎，需重保。
2. **LLM分组：**承担基于不同业务场景的大模型实时推理核心职能，动态调度多类型Agent数据生产链路；除实时推理任务外，离线推理任务也同样支持；在此分组上支持大模型评测

体系的任务，辅助提升业务推理数据的质量。

3. **服务分组：**采用读写分离+单元化架构实现Agent数据的高效写入和投放：

- 写服务：以张北中心为唯一写入节点，通过`dh_L1`服务层与双机房失效缓存机制，确保数据写入的集中管控与容灾能力，所有写操作最终持久化至中心数据库（DB）；
- 读服务：按照详情等单元化流量进行部署，通过`dh_L0d`服务层、Tair分布式缓存（NA610/NA620）及单元化读缓存链路，实现低延迟数据访问。跨中心数据通过"单元同步"机制保障最终一致性，满足海量读请求的就近访问需求。

小结

相较于引入功能完备但复杂Agent框架或者试图在Java领域复刻一个完整的平台级框架，我们只希望拥有一个“恰好够用”但又不会带来太多负担的SDK。这样的SDK仅需要cursor两个小时的对话以及2天的测试打磨即可，事实上目前的aiagentsdk几乎100%都是AI生成的代码。至此，我们初步搭建完成了以“事件驱动的Function-Centric Agent架构”。

04 商品Agent应用进展

商品Agent已在商品属性、卖点等多个核心业务场景完成落地，覆盖亿级在线商品，显著提升了商品信息的完整性、准确率与丰富性，并在搜索、详情等核心导购场景中实现了成交转化率的正向提升。

在研发效能层面，面向数据流处理的商品Agent基础框架已全面搭建完成，具备较强的能力复用性与快速扩展能力。新业务需求到来时，开发团队仅需一周/人的时间即可完成能力开发、数据试跑及评测结果优化等全流程工作，大幅压缩了从需求到前台实验的交付周期，助力业务以更快的节奏在前台场景完成验证与迭代。

05 总结

通过25年一个季度对商品域Agent的探索与建设，我们初步验证了将大模型能力以结构化、工程化方式嵌入商品数据生产链路的可行性与价值。从底层事务驱动的数据引擎升级，到轻量Agent架构设计，再到多个核心业务场景的成功落地，商品Agent已展现出从“单点提效”向“系统自治”演进的巨大潜力。

展望未来，Agent技术正以前所未有的速度迭代演进，Harness、Skill等新一代Agent框架与范式的持续涌现，为商品域的智能化建设打开了全新的想象空间。我们正积极拥抱这些前沿

技术，将其与商品域的实际业务场景深度融合：以更强的语义理解能力让Agent真正"读懂"商品的深层含义，以Skill模块化的方式沉淀和复用商品域的专业领域知识，以商品理解"大脑"为核心构建自适应决策机制，实现对商品信息的主动感知与动态更新。我们相信，持续夯实"数据+工具+知识+决策"四位一体的Agent基础设施，在拥抱技术浪潮中驱动能力持续跃迁，让系统真正读懂每一件商品、理解每一位消费者，驱动电商导购体验进入“主动理解、精准表达、高效决策”的新阶段。



本文作者勇清，来自淘天集团–商品中心技术团队。在海量商品构成的复杂生态下，我们系统性地构建了贯穿商品定义、发布、存储、治理、理解与消费分发的全生命周期管理能力，帮助商家与平台实现商品信息的标准化表达、智能化理解与规模化流通。我们的愿景，是让商品数据如同血液一般在集团内健康循环、畅通流转、智慧运行——以集团统一的商品数据底座与商品理解 Agent 为核心驱动力，全力支撑淘宝、天猫等核心电商业务线的持续发展与创新。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 7641

留言

写留言

还没有留言，来写首评